

SoSe 26 Computer Graphics Solutions

Igor Dimitrov

2026-04-21

Table of contents

Preface	3
1 Sheet 1	4
1.1 Assignment 1.1	4
1.2 Assignment 1.2	4
2 Sheet 2	6
2.1 Assignment 1	6
3 2021 solutions	11
3.1 Task 1	11
3.1.1 Task 1(h)	13

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Sheet 1

Cemile Buyukakkac

Malte Herzog

Igor Dimitrov

1.1 Assignment 1.1

a) $|\text{Pixel}| = 3840 \times 2160 = 829440$

The whole display is generated in $t = \frac{1}{f} = \frac{1}{60} s$

$\Rightarrow$ Generation of a single pixel can take at most:

$$t_{max} = \frac{1}{3840 \times 2160 \times 60} s \approx 2ns$$

b) Data load of 24 bits per pixel is due to three color channels Red, Green, and Blue, each 8 bit, i.e. 1B. Therefore there are 8 Bytes per pixel and 3840×2160 pixels in total. Since the display is regenerated every $1/60$ seconds, we have:

$$B = 3B \times 3840 \times 2160 \times 60 \frac{1}{s} \\ \approx 1.4GB/s$$

1.2 Assignment 1.2

a) There are three types of light-sensitive receptors,

- S: corresponding to blue
- M: corresponding to green
- L corresponding to red but the perception of color is calculated simply as superposition of three channels but rather as two channels combination of pairs of opposing colors

- b) To focus (or zoom in this context) means to reduce the field of vision, and reducing the field of vision corresponds to a smaller solid angle. The smaller the solid angle is, the less light arrives at the camera lens or human eye through the telescope, which makes the object appear dimmer, or not bright enough.
- c) metamers are color pairs that look or are perceived identically to the human eye, although they emit different light spectra, because they produce the same response in the three cone types (L, M, S)

2 Sheet 2

2.1 Assignment 1

- a)
 - Vector-based advantages:
 - resolution independent
 - Vector-based disadvantages:
 - image build-up is difficult in complex scenes
 - Raster-based advantages:
 - repeated image build-up independent of scene complexity
 - Raster-based disadvantages:
 - Moire effects due to finite number of pixels
- b) a spectral color, or a monochromatic color, is light produced by the light of single wavelength in the visible spectrum
- c)
- d) mach bands are an optical illusion where we perceive extra-bright and extra-dark bands near the boundary between two regions with different luminance gradients

a receptive field have usually a center-surround structure where light in the center excites a neuron, while the light in the surrounding region inhibits it, or vice versa.

Mach bands arise because these center-surround receptive fields perform a local contrast enhancement. Near a luminance edge, one receptive field may receive more excitation than inhibition, making the side look brighter, or vice versa. e) three operations: i)

relation to YIQ

in the YIQ model the Y channel is calculated approximately as:

$$Y = 0.3 R + 0.6G + 0.1 B$$

where we see that green contributes the most and blue the least. This corresponds roughly to the human perception of brightness - it is strongest around green-yellow region and weakest in the blue region, due to S cells contributing close to nothing to the perception of brightness.

I and Q channels carry color-difference information. They do not correspond exactly to the biological opponent channels, but they similarly separate color information from brightness detail - human eye is much more sensitive to contrast than to color difference

1) The light spectrum is given as

$$P(\lambda) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\lambda - \mu)^2}{2\sigma^2}\right) W$$

Is a Gaussian / normal distribution shape:

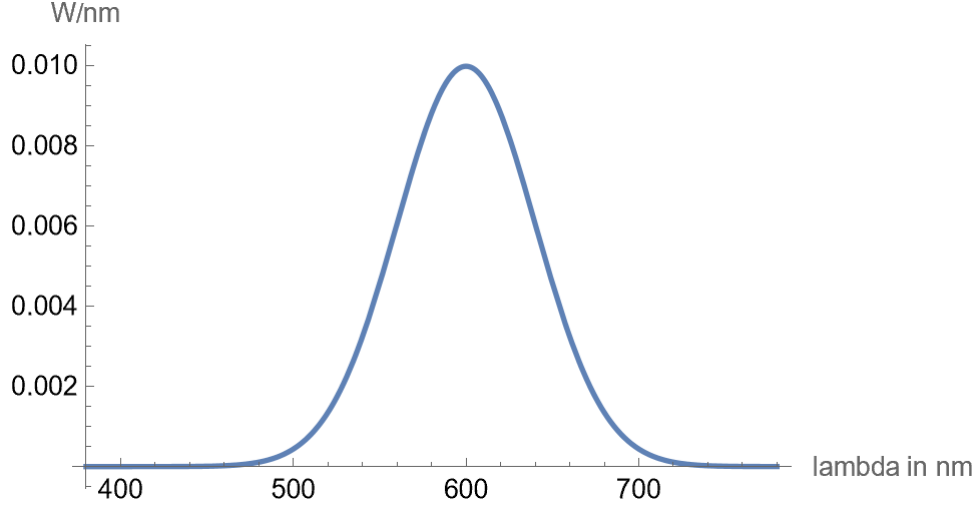


Figure 2.1: $P(\lambda)$

And the color matching functions are given as:

$$\bar{x}(\lambda) = 1.065 \exp\left(-\frac{1}{2} \left(\frac{\lambda - 595.8}{33.33}\right)^2\right) + 0.366 \exp\left(-\frac{1}{2} \left(\frac{\lambda - 446.8}{19.44}\right)^2\right)$$

$$\bar{y}(\lambda) = 1.014 \exp\left(-\frac{1}{2} \left(\frac{\ln \lambda - \ln 556.3}{0.075}\right)^2\right)$$

$$\bar{z}(\lambda) = 1.839 \exp\left(-\frac{1}{2} \left(\frac{\ln \lambda - \ln 449.8}{0.051}\right)^2\right)$$

which can be plotted as follows (together with a scaled version of $P(\lambda)$):

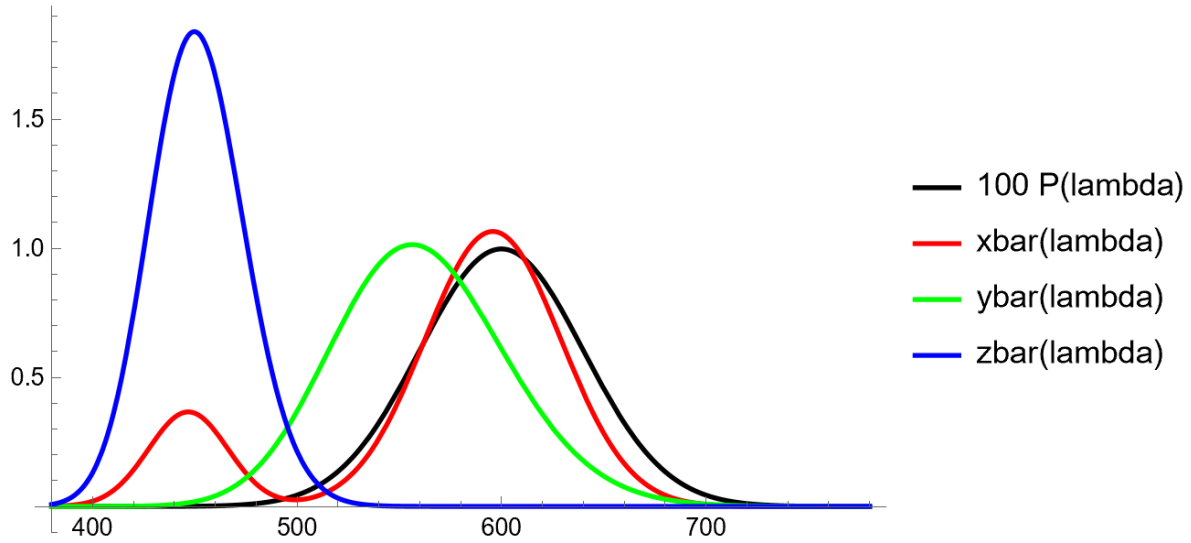


Figure 2.2: x, y, z

To compute X, Y, Z we compute the integrals:

$$X = \int_{\lambda} \bar{x}(\lambda) P(\lambda) d\lambda,$$

$$Y = \int_{\lambda} \bar{y}(\lambda) P(\lambda) d\lambda,$$

$$Z = \int_{\lambda} \bar{z}(\lambda) P(\lambda) d\lambda.$$

We can use a symbolic or numerical mathematics software. In our case we used mathematica where we computed the integrals with the following code:

First we define the functions:

```
sigma = 40;
mu = 600;

P[lambda_] := 1/Sqrt[2 Pi sigma^2] Exp[-(lambda - mu)^2/(2 sigma^2)];

xbar[lambda_] :=
  1.065 Exp[-1/2 ((lambda - 595.8)/33.33)^2] +
  0.366 Exp[-1/2 ((lambda - 446.8)/19.44)^2];

ybar[lambda_] :=
```



```

1.014 Exp[-1/2 ((Log[lambda] - Log[556.3])/0.075)^2];

zbar[lambda_] :=
1.839 Exp[-1/2 ((Log[lambda] - Log[449.8])/0.051)^2];

```

Then we compute the integrals with:

```

X = NIntegrate[xbar[lambda] P[lambda], {lambda, 380, 780}];
Y = NIntegrate[ybar[lambda] P[lambda], {lambda, 380, 780}];
Z = NIntegrate[zbar[lambda] P[lambda], {lambda, 380, 780}];

```

and find

$$\begin{aligned}
X &\approx 0.679966 \\
Y &\approx 0.569252 \\
Z &\approx 0.00561767
\end{aligned}$$

and finally we compute the x, y values as:

$$\begin{aligned}
x &= \frac{X}{X+Y+Z} \approx 0.541881 \\
y &= \frac{Y}{X+Y+Z} \approx 0.453642
\end{aligned}$$

To compute linear RGB from XYZ we use the following linear transformation:

$$\begin{pmatrix} R_{\text{lin}} \\ G_{\text{lin}} \\ B_{\text{lin}} \end{pmatrix} = \begin{pmatrix} 3.2406 & -1.5372 & -0.4986 \\ -0.9689 & 1.8758 & 0.0415 \\ 0.0557 & -0.2040 & 1.0570 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}.$$

We achieve this with the following mathematica code:

```

XYZtoLinearSRGB[{X_, Y_, Z_}] := {
  3.2406 X - 1.5372 Y - 0.4986 Z,
  -0.9689 X + 1.8758 Y + 0.0415 Z,
  0.0557 X - 0.2040 Y + 1.0570 Z
}

{R, G, B} = XYZtoLinearSRGB[{X, Y, Z}]

```

and obtain:

$$\begin{aligned}R &\approx 1.33 \\G &\approx 0.41 \\B &\approx -0.07\end{aligned}$$

Here R and B are outside of representable RGB gamut. Clipping them we get

$$\begin{aligned}R &\approx 1 \\G &\approx 0.41 \\B &\approx 0\end{aligned}$$

Converting this to $[0, 255]$ scale we get

$$(R, G, B) = (255, 171, 0)$$

This is a bright yellow-orange color, with no blue

3 2021 solutions

3.1 Task 1

- a) raster graphics vs vector graphics:
- b) explain & motivate following:

- i) Up vector:

The **up vector** specifies which direction should appear upward in the camera image. Together with the viewing direction, it determines the camera coordinate system and removes the otherwise undetermined rotation around the viewing axis (camera roll). It must not be parallel to the viewing direction.

- ii) **Double buffering:**

Yes—the main relevant slide is **CG06_1, slide 129**; slide 130 extends it to stereoscopic rendering.

Double buffering: Two color buffers are used. The **front buffer** contains the complete image currently displayed, while the next frame is rendered invisibly into the **back buffer**. When rendering is finished, the buffers are swapped. This prevents incomplete, partially rendered images from being displayed and decouples rendering from the screen refresh.

- iii)

- iv) **backface culling: Backface culling:** For each triangle, its front and back faces are distinguished, commonly using the order of its projected vertices: counterclockwise winding usually indicates a front face. Triangles facing away from the camera are discarded before rasterization. For closed meshes, these back faces are normally hidden by front faces anyway, so culling them avoids unnecessary rendering work. It must be disabled or adjusted when viewing a mesh from the inside or rendering two-sided surfaces.

- c) explain how vertex normals can be computed from a given mesh. In that context explain the issues that can arise from degenerate triangles

For every triangle, compute its face normal using the cross product of two edges,

$$\mathbf{n}_f = (\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1).$$

For each vertex, sum the normals of all adjacent faces—optionally weighted by their areas or corner angles—and normalize the resulting vector to obtain the vertex normal.

If a triangle has a zero-length edge, its cross product is the zero vector. This vector cannot be normalized and may produce undefined values that contaminate the vertex-normal calculation. Such degenerate triangles should therefore be detected and ignored or removed.

- d) rendering sharp edges can be done by introducing multiple normal per vertex. Does this work equally well for gouraud and phong

The approach works for both methods, but not with equal shading quality:

- **Gouraud shading:** Illumination is calculated at each vertex. The different normals assigned to the same geometric position produce different vertex colors for the adjacent faces. These colors are interpolated separately within each face, preserving the discontinuity at the sharp edge. However, lighting effects between vertices—especially small specular highlights—may be missed.
- **Phong shading:** The separate normals are interpolated within their respective faces, and illumination is calculated for every fragment. The normal remains discontinuous across the edge, while lighting within each face is evaluated more accurately.

Thus, both can represent sharp edges, but Phong shading generally renders their illumination more accurately, at greater computational cost. [Phong's per-pixel shading formulation](#) also reflects this accuracy-cost trade-off.

- e) two projection types, explain the difference:

The two widely used projection types are:

- **Orthographic (parallel) projection:** Projection rays are parallel. An object's apparent size does not decrease with distance, and parallel lines remain parallel.
- **Perspective projection:** Projection rays meet at the camera's projection centre. Distant objects appear smaller, producing perspective foreshortening and a more realistic impression of depth.

- f) explain the reasons for aliasing and artifact it leads to

Aliasing is caused by sampling a continuous image or signal at an insufficient spatial or temporal frequency. In particular, if the sampling frequency is not greater than twice the highest signal frequency, high-frequency details are incorrectly represented as lower, artificial frequencies. Sharp object boundaries are especially problematic because they contain very high frequencies.

Typical artifacts include **jagged or staircase-shaped edges**, **Moiré patterns**, distorted fine details, and flickering or crawling patterns in animations.

g) role of perspective division and its relation to dehomogenization

After the projection transformation, a point is represented in homogeneous clip coordinates as

$$(x_h, y_h, z_h, w_h).$$

Perspective division divides the first three coordinates by (w_h):

$$(x_h, y_h, z_h, w_h) \mapsto \left(\frac{x_h}{w_h}, \frac{y_h}{w_h}, \frac{z_h}{w_h} \right).$$

Since (w_h) depends on the point's depth, this division makes distant objects appear smaller and maps the perspective view frustum to the normalized unit cube.

Perspective division is therefore precisely the **dehomogenization** performed after the projection transformation; in the course, this complete step is also called the **normalization transformation**.

h) rendering equation

- i) explain different parts of the rendering equation
- ii) how if f typically named, how many parameters (dimensions) does it have?
- iii) how many parameters, dimensions does the isotropic version of f have
- iv) explain why this equations

3.1.1 Task 1(h)

The rendering equation is

(i) Components

- **(A)** $L(\mathbf{x}, \omega)$: total outgoing radiance leaving point $\mathbf{x}$ in direction ω .
- **(B)** $E(\mathbf{x}, \omega)$: radiance emitted by the surface itself in direction ω .
- **(C)** $f(\mathbf{x}, \omega_i \rightarrow \omega)$: describes what proportion of light arriving from ω_i is reflected toward ω . The integral sums reflected light arriving from all directions in the positive hemisphere Ω^+ . The factor $\cos \theta_i$ accounts for the smaller effective surface area presented to light arriving at a grazing angle.

(ii) f is called the **bidirectional reflectance distribution function (BRDF)**. At a fixed surface point, it is four-dimensional: two angles specify the incoming direction and two specify the outgoing direction,

$$f(\theta_i, \phi_i, \theta_o, \phi_o).$$

(iii) An isotropic BRDF is invariant under simultaneous rotation of both directions around the surface normal. Only the relative azimuth matters, so it is three-dimensional:

$$f(\theta_i, \theta_o, \phi_i - \phi_o).$$

(iv) The equation is recursive because the incoming radiance at $\mathbf{x}$ is outgoing radiance from another visible surface point. That outgoing radiance is itself determined by the rendering equation and depends on light arriving there from still other surfaces. In ray tracing, this leads to recursively tracing secondary rays until a light source is reached or a termination condition is applied.